```python
# Use a standalone function when there is no persistent object state.
"""
Rating Rationale section
├── Child: paragraph 1
├── Child: paragraph 2
└── Child: paragraph 3
"""


def hierarchical_chunking(document_id, section, paragraphs):
    chunks = []
    parent_id = f"{document_id}-{section}"

    chunks.append({
        "chunk_id": f"{parent_id}",
        "doc_id": f"{document_id}-{section}",
        "chunk_type": "parent",
        "parent_id": None,
        "text": "\n".join(paragraphs),
    })

    for index, paragraph in enumerate(paragraphs):
        chunks.append({
            "chunk_id": f"{parent_id}-{index}",
            "doc_id": f"{document_id}-{section}",
            "chunk_type": "child",
            "parent_id": f"{parent_id}",
            "text": paragraph,
        })

    return chunks


def table_chunks(document_id, table_name, rows):
    chunks = []

    for index, row in enumerate(rows):
        row_text = " | ".join(
            f"{column}:{value}" for column, value in row.items()
        )

        chunks.append({
            "chunk_id": f"{document_id}-{table_name}-{index}",
            "doc_id": f"{document_id}-{table_name}",
            "row_number": index,
            "text": f"{table_name}|{row_text}",
        })

    return chunks


def flat_chunks(doc_id, text, chunk_size=100, overlap=20):
    chunks = []
    words = text.split()
    step = chunk_size - overlap

    if step <= 0:
        raise ValueError("overlap must be smaller than chunk_size")

    for index, start in enumerate(range(0, len(words), step)):
        chunk_text = " ".join(words[start:start + chunk_size])

        chunks.append({
            "chunk_id": f"{doc_id}-flat-{index}",
            "doc_id": doc_id,
```

```
                "chunk_type": "flat",
                "parent_id": None,
                "text": chunk_text,
            })

    return chunks
```